wintime_vignette

wintime Package Guide

This vignette demonstrates the use of the three available functions contained in the wintime package: `wintime`, `markov`, and `km`. The `wintime` function is the main function of the package that implements the methods described in “Use of win time for ordered composite endpoints in clinical trials,” (Troendle et al. 2024), https://pubmed.ncbi.nlm.nih.gov/38417455/. This function accepts data with any number of endpoints and runs these methods on observed and resampled data. The `markov` function fits an extended Markov model to estimate the state probabilities for each arm. The `km` function performs the same estimation using Kaplan-Meier models.

Installation

To use these functions, you need to have the `wintime` package installed. You can install it from CRAN using the `install.packages(wintime)` command.

Load Required Packages

Load the `wintime` package along with the `survival` package.

```
library(wintime)
library(survival)
```

Example Data

In this section, we will create example data to demonstrate the use of the package functions. The data includes event times, event indicators, treatment arm indicators, and optional covariates. We will consider a dataset with 3 endpoints ordered into a composite. In this dataset, the event time vectors are titled `TIME_1`, `TIME_2`, and `TIME_3`. These must be organized into a matrix with rows representing events and columns representing participants. It is critical that event rows are in INCREASING order of severity. The event indicator vectors, titled `DELTA_1`, `DELTA_2`, and `DELTA_3`, will be organized the same way as the event times. The treatment arm indicator vector, titled `trt`, will be entered as a vector containing 1’s and 0’s (control = 0, treatment = 1). Covariates are optional, but if they are entered into the `wintime` function, they must also be entered as a matrix. The three covariates in this dataset, titled `cov1`, `cov2`, and `cov3`, will be organized into a matrix with rows representing participants and columns representing covariate values.

```
# Event time vectors
TIME_1 <- c(256,44,29,186,29,80,11,380,102,33)
TIME_2 <- c(128,44,95,186,69,66,153,380,117,33)
TIME_3 <- c(435,44,95,186,69,270,1063,380,117,33)

# Event time matrix
Time <- rbind(TIME_1, TIME_2, TIME_3)
```

```r
# Event indicator vectors
DELTA_1 <- c(1,0,1,0,1,1,1,0,1,0)
DELTA_2 <- c(1,0,0,0,0,1,1,0,0,0)
DELTA_3 <- c(0,0,0,0,0,0,0,0,0,0)

# Event indicator matrix
Delta <- rbind(DELTA_1, DELTA_2, DELTA_3)

# Treatment arm indicator vector
trt <- c(1,1,1,1,1,0,0,0,0,0)

# Covariate vectors
cov1 <- c(66,67,54,68,77,65,55,66,77,54)
cov2 <- c(3,6,4,2,3,5,8,5,3,5)
cov3 <- c(34.6,543.6,45.8,54.7,44.3,55.6,65.9,54.7,77.9,31.2)

# Covariate matrix
cov <- cbind(cov1, cov2, cov3)

cat("Time =", "\n")


## Time =


print(Time)


##        [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## TIME_1  256   44   29  186   29   80   11  380  102    33
## TIME_2  128   44   95  186   69   66  153  380  117    33
## TIME_3  435   44   95  186   69  270 1063  380  117    33


cat("Delta =", "\n")


## Delta =


print(Delta)


##         [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## DELTA_1    1    0    1    0    1    1    1    0    1     0
## DELTA_2    1    0    0    0    0    1    1    0    0     0
## DELTA_3    0    0    0    0    0    0    0    0    0     0


cat("trt", "\n")
```

```
## trt
```

```
print(trt)
```

```
##  [1] 1 1 1 1 1 0 0 0 0 0
```

```
cat("cov =", "\n")
```

```
## cov =
```

```
print(cov)
```

```
##       cov1 cov2  cov3
##  [1,]   66    3  34.6
##  [2,]   67    6 543.6
##  [3,]   54    4  45.8
##  [4,]   68    2  54.7
##  [5,]   77    3  44.3
##  [6,]   65    5  55.6
##  [7,]   55    8  65.9
##  [8,]   66    5  54.7
##  [9,]   77    3  77.9
## [10,]   54    5  31.2
```

wintime Function

The required arguments for the `wintime` function are the desired win time method, a matrix of event times, a matrix of event indicators, and a vector of treatment arm indicators. The win time method is entered as a string value indicating which method will be run. Methods include: `wtr`, `rwtr`, `pwt`, `ewtr`, `ewt`, `max`, and `rmt`. This section will demonstrate the use of each method with default settings. Each of these function calls will return a list with seven elements: the observed treatment effect, a vector of resampled treatment effects (this will be empty with no resampling), a message describing which method was used, the variance, the p-value, the number of wins for treatment (pairwise methods only), and the number of losses for treatment (pairwise methods only).

Win time ratio (wtr)

```
# Run wtr
result <- wintime("wtr", Time, Delta, trt)
print(result)
```

```
## $data
## [1] 1
##
```

```
## $resample_data
## numeric(0)
##
## $message
## [1] "Resampling of  wtr  done on  0  bootstraps."
##
## $variance
## [1] NA
##
## $p
## [1] NA
##
## $wins
## [1] 9
##
## $losses
## [1] 9
```

Restricted win time ratio (rwtr)

```
# Run rwtr
result <- wintime("rwtr", Time, Delta, trt)
print(result)
```

```
## $data
## [1] 1
##
## $resample_data
## numeric(0)
##
## $message
## [1] "Resampling of  rwtr  done on  0  bootstraps."
##
## $variance
## [1] NA
##
## $p
## [1] NA
##
## $wins
## [1] 9
##
## $losses
## [1] 9
```

Pairwise win time (pwt)

```r
# Run pwt
result <- wintime("pwt", Time, Delta, trt)
print(result)
```

```
## $data
## TIME_1
##   1.36
##
## $resample_data
## numeric(0)
##
## $message
## [1] "Resampling of  pwt  done on  0  bootstraps."
##
## $variance
## [1] NA
##
## $p
## TIME_1
##     NA
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

Expected win time against reference (ewtr)

The ewtr method accepts a matrix of covariates as an optional argument. If you wish to enter covariates, pass it as the value of `cov`.

```r
# Run ewtr with covariates
result <- wintime("ewtr", Time, Delta, trt, cov = cov)
print(result)
```

```
## $data
##       trt
## -150.9492
##
## $resample_data
## [1] NA
##
## $message
## NULL
##
## $variance
## [1] 14858.91
```

```
##
## $p
##       trt
## 0.2155928
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

Expected win time (ewt)

```
# Run ewt
result <- wintime("ewt", Time, Delta, trt)
print(result)
```

```
## $data
## [1] -37.45417
##
## $resample_data
## numeric(0)
##
## $message
## [1] "Resampling of  ewt  done on  0  permutations."
##
## $variance
## [1] NA
##
## $p
## [1] 1
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

EWTR-composite max test (max)

```
# Run max
result <- wintime("max", Time, Delta, trt, cov = cov)
print(result)
```

```
## $data
## [1] -1.238333
```

```
##
## $resample_data
## numeric(0)
##
## $message
## [1] "Resampling of  max  done on  0  bootstraps."
##
## $variance
## [1] NA
##
## $p
## [1] 1
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

Restricted mean survival in favor of treatment (rmt)

For this method, the `rmst_restriction` argument must be passed. This is the cutoff time for the rmt method.

```
# Run rmt
result <- wintime("rmt", Time, Delta, trt, rmst_restriction = round(1.5*365.25))
print(result)
```

```
## $data
## [1] -37.45417
##
## $resample_data
## numeric(0)
##
## $message
## [1] "Resampling of  rmt  done on  0  permutations."
##
## $variance
## [1] NA
##
## $p
## [1] 1
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

Resampling

In this section, we will demonstrate the use of resampling with a few of the win time methods. To run a win time method with resampling, just pass the `resample_num` argument as an integer value that tells the `wintime` function how many times you wish to resample. Each method is either resampled with bootstraps or permutations by default. The third element of the returned list will indicate which resampling technique was used. Resampling involves random number generation, so we can pass the optional `seed` argument for replicability.

ewt with 10 Resamples

```
# Run ewt with 10 resamples
result <- wintime("ewt", Time, Delta, trt, resample_num = 10, seed = 123)
print(result)
```

```
## $data
## [1] -37.45417
##
## $resample_data
##  [1] -26.24444  27.36667 -10.50000 -42.28000 -39.88000 -20.44000 -25.80000
##  [8] -69.53333 -25.80000 -46.50556
##
## $message
## [1] "Resampling of  ewt  done on  10  permutations."
##
## $variance
## [1] 650.4605
##
## $p
## [1] 0.6363636
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

wtr with 5 Resamples

It is important to note that `wtr` is a pairwise method. So, with a large number of resamples, it could take several minutes to complete execution.

```
# Run wtr with 5 resamples
result <- wintime("wtr", Time, Delta, trt, resample_num = 5)
print(result)
```

```
## $data
## [1] 1
##
## $resample_data
## [1] 1.777778 6.000000 2.666667 1.000000 1.000000
##
## $message
## [1] "Resampling of  wtr  done on  5  bootstraps."
##
## $variance
## [1] 4.324691
##
## $p
## [1] 1
##
## $wins
## [1] 9
##
## $losses
## [1] 9
```

Resampleing Cont.

As we mentioned above, each win time method is set up with a default resampling technique. This can be overridden by passing the optional `resample` argument. However, this is not recommended. By changing the resampling technique, a user will receive a warning message.

For example, if we want to run the ewt method with bootstraps, we can do so by passing “boot” as the value for `resample`.

```
# Run ewt on 10 bootstraps
result <- wintime("ewt", Time, Delta, trt, resample_num = 10, resample = "boot")
```

```
## Warning in wintime("ewt", Time, Delta, trt, resample_num = 10, resample =
## "boot"): For this method, it is strongly recommended to use a KM model and to
## resample using permutations. These are set as defaults.
```

```
print(result)
```

```
## $data
## [1] -37.45417
##
## $resample_data
##  [1] -33.583333 -21.285714 -36.000000   7.518519  48.666667 -24.666667
##  [7] -36.083333   8.180952 -41.809524 -36.190476
##
## $message
```

```
## [1] "Resampling of  ewt  done on  10  bootstraps."
##
## $variance
## [1] 845.1343
##
## $p
## [1] 0.9090909
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

Survival Models

The `wintime` package contains two functions that estimate the state probabilities for each arm. These will be demonstrated in the next section. Each non-pairwise method requires an estimation of the state probabilities. These are calculated using either a Markov model or a Kaplan-Meier model. Because each method is set up with a default model, no input is required. Although, if a user wishes to use one over the other, they may do so by passing the optional `model` argument into the `wintime` function. This will trigger a warning message indicating that this combination of `type` and `model` is not recommended.

For example, if we want to run the ewtr method using a Kaplan-Meier model, we can do so by passing “km” as the value for `model`.

```
# Run ewtr with a KM model
result <- wintime("ewtr", Time, Delta, trt, cov = cov, model = "km")
```

```
## Warning in wintime("ewtr", Time, Delta, trt, cov = cov, model = "km"): For this
## method, it is strongly recommended to use a Markov model. This is set as a
## default.
```

```
print(result)
```

```
## $data
##      trt
## -108.094
##
## $resample_data
## [1] NA
##
## $message
## NULL
##
## $variance
## [1] 12739.96
##
```

```
## $p
##       trt
## 0.3382273
##
## $wins
## [1] NA
##
## $losses
## [1] NA
```

Model Functions

To use either the `markov` or `km` functions, we need the number of control arm patients, the number of treatment arm patients, the number of endpoints, a matrix of event times, and a matrix of event indicators. Both functions return a list with eight elements: a matrix of control arm state probabilities, a matrix of treatment arm state probabilities, a vector of unique control arm event times, a vector of unique treatment arm event times, the number of unique control arm event times, the number of unique treatment arm event times, the control arm max follow time, and the treatment arm max follow time.

Parameters

The event time and indicator matrices are already defined as `Time` and `Delta` (see 'Example Data' section above). The treatment arm indicator vector, `trt`, is not required for these functions, but it is useful in defining other parameters. This is also defined in the 'Example Data' section.

```
# Number of control arm patients
n0 <- sum(trt == 0)

# Number of treatment arm patients
n1 <- sum(trt == 1)

# Number of endpoints
m <- nrow(Time)
```

markov Function

```
# Run markov
result <- markov(n0, n1, m, Time, Delta)

# Control arm probabilities
dist0 <- result[[1]]
cat("dist0 =", "\n")
```

```
## dist0 =
```

```r
print(dist0)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,]  0.6  0.6  0.6  0.6  0.3  0.3  0.3
## [2,]  0.4  0.4  0.4  0.4  0.4  0.4  0.4
## [3,]  0.0  0.0  0.0  0.0  0.3  0.3  0.3
## [4,]  0.0  0.0  0.0  0.0  0.0  0.0  0.0
```

```r
# Treatment arm probabilities
dist1 <- result[[2]]
cat("dist1 =", "\n")
```

```
## dist1 =
```

```r
print(dist1)
```

```
##      [,1] [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]
## [1,]  0.8  0.8 0.5333333 0.2666667 0.2666667 0.2666667 0.2666667 0.2666667
## [2,]  0.2  0.2 0.2000000 0.4666667 0.4666667 0.0000000 0.0000000 0.0000000
## [3,]  0.0  0.0 0.2666667 0.2666667 0.2666667 0.7333333 0.7333333 0.7333333
## [4,]  0.0  0.0 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
##           [,9]
## [1,] 0.2666667
## [2,] 0.0000000
## [3,] 0.7333333
## [4,] 0.0000000
```

```r
# Unique control arm event times
untimes0 <- result[[3]]
cat("untimes0 =", "\n")
```

```
## untimes0 =
```

```r
print(untimes0)
```

```
## [1]  29  44  69  95 128 186 435
```

```r
# Unique treatment arm event times
untimes1 <- result[[4]]
cat("untimes1 =", "\n")
```

```
## untimes1 =
```

```r
print(untimes1)
```

```
## [1]   11   33   66  102  117  153  270  380 1063
```

```r
# Number of unique control arm event times
nuntimes0 <- result[[5]]
cat("nuntimes0 =", "\n")
```

```
## nuntimes0 =
```

```r
print(nuntimes0)
```

```
## [1] 7
```

```r
# Number of unique treatment arm event times
nuntimes1 <- result[[6]]
cat("nuntimes1 =", "\n")
```

```
## nuntimes1 =
```

```r
print(nuntimes1)
```

```
## [1] 9
```

```r
# Control arm max follow time
max_follow0 <- result[[7]]
cat("max_follow0 =", "\n")
```

```
## max_follow0 =
```

```r
print(max_follow0)
```

```
## [1] 95
```

```r
# Treatment arm max follow time
max_follow1 <- result[[8]]
cat("max_follow1 =", "\n")
```

```
## max_follow1 =
```

```
print(max_follow1)
```

```
## [1] 380
```

km Function

```
# Run km
result <- km(n0, n1, m, Time, Delta)

# Control arm probabilities
dist0 <- result[[1]]
cat("dist0 =", "\n")
```

```
## dist0 =
```

```
print(dist0)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13] [,14]
## [1,]  0.6  0.6  0.6  0.6  0.3  0.3    0    0    0     0     0     0     0     0
## [2,]  0.4  0.4  0.4  0.4  0.2  0.2    0    0    0     0     0     0     0     0
## [3,]  0.0  0.0  0.0  0.0  0.5  0.5    0    0    0     0     0     0     0     0
## [4,]  0.0  0.0  0.0  0.0  0.0  0.0    0    0    0     0     0     0     0     0
```

```
# Treatment arm probabilities
dist1 <- result[[2]]
cat("dist1 =", "\n")
```

```
## dist1 =
```

```
print(dist1)
```

```
##      [,1] [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8] [,9]
## [1,]  0.8  0.8 0.5333333 0.2666667 0.2666667 0.2666667 0.2666667 0.2666667    0
## [2,]  0.2  0.2 0.2166667 0.4833333 0.4833333 0.1083333 0.1083333 0.1083333    0
## [3,]  0.0  0.0 0.2500000 0.2500000 0.2500000 0.6250000 0.6250000 0.6250000    0
## [4,]  0.0  0.0 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000    0
##      [,10] [,11] [,12] [,13] [,14] [,15]
## [1,]     0     0     0     0     0     0
## [2,]     0     0     0     0     0     0
```

```
## [3,]    0    0    0    0    0    0
## [4,]    0    0    0    0    0    0
```

```
# Unique control arm event times
untimes0 <- result[[3]]
cat("untimes0 =", "\n")
```

```
## untimes0 =
```

```
print(untimes0)
```

```
## [1]  29  44  69  95 128 186
```

```
# Unique treatment arm event times
untimes1 <- result[[4]]
cat("untimes1 =", "\n")
```

```
## untimes1 =
```

```
print(untimes1)
```

```
## [1]  11  33  66 102 117 153 270 380
```

```
# Number of unique control arm event times
nuntimes0 <- result[[5]]
cat("nuntimes0 =", "\n")
```

```
## nuntimes0 =
```

```
print(nuntimes0)
```

```
## [1] 6
```

```
# Number of unique treatment arm event times
nuntimes1 <- result[[6]]
cat("nuntimes1 =", "\n")
```

```
## nuntimes1 =
```

```r
print(nuntimes1)
```

```
## [1] 8
```

```r
# Control arm max follow time
max_follow0 <- result[[7]]
cat("max_follow0 =", "\n")
```

```
## max_follow0 =
```

```r
print(max_follow0)
```

```
## [1] 186
```

```r
# Treatment arm max follow time
max_follow1 <- result[[8]]
cat("max_follow1 =", "\n")
```

```
## max_follow1 =
```

```r
print(max_follow1)
```

```
## [1] 380
```